

CYBERDESK

Projekt-Handbuch // Cyberpunk Desktop Simulation

Ein dystopisches Desktop-Erlebnis in PyQt6
Kursprojekt GUI – 64 Stunden – 5 Entwickler

Technologie	Python 3.11+ / PyQt6
Architektur	MVC + EventBus (Publish/Subscribe)
Team	5 Entwickler, rotierende Rollen
Phasen	4 × 16 Stunden
Versionierung	Git, Feature-Branch-Workflow
Code-Stil	PEP8, Type Hints, Docstrings

Inhaltsverzeichnis

1.	Projektüberblick & Vision
2.	Architektur & Designentscheidungen
3.	Ordnerstruktur
4.	Rollen & Rotation
5.	Phasen-Ablaufplan (64h)
6.	Detaillierter Stunden-Plan
7.	Coding Standards & Clean Code
8.	Git-Workflow
9.	Starter-Code: Kernsystem
10.	App-Spezifikationen
11.	Datenmodelle & Persistenz
12.	Cyberpunk Look & Feel
13.	Easter Eggs
14.	Tests & Qualitätssicherung
15.	Risiken & Notfallplan
16.	Abnahme-Checkliste

1. Projektüberblick & Vision

CyberDesk ist **kein Betriebssystem** – es ist eine **Simulation** eines Desktops aus einer dystopischen Cyberpunk-Welt. Der Nutzer öffnet die Anwendung und erlebt einen funktionsfähigen Desktop mit Fenstern, Drag & Drop, einem Terminal, einem Datei-Explorer und weiteren Apps – alles eingebettet in eine düstere Neon-Ästhetik. Persistenz ist über JSON gegeben, sodass der Zustand zwischen Sitzungen erhalten bleibt.

Designziele

- **Funktional:** Ein echtes Fenstersystem mit mehreren parallelen Apps.
- **Atmosphärisch:** Cyberpunk-Look durch QSS-Stylesheets, Sounds, Fonts.
- **Wartbar:** Klare Architektur, sodass jeder im Team ohne Konflikte arbeiten kann.
- **Erweiterbar:** Neue Apps lassen sich durch Erben von BaseApp hinzufügen.
- **Lehrreich:** Jeder im Team berührt jeden Bereich – kein Vibe Coding.

Was wir NICHT bauen

- Kein echtes Betriebssystem, kein Kernel, keine Treiber.
- Keine echte Netzwerkkommunikation – alles ist simuliert.
- Keine externen Programme starten – nur unsere eigenen Apps.

2. Architektur & Designentscheidungen

Wir verwenden ein **MVC-ähnliches Pattern** in Kombination mit einem **EventBus** (Publish/Subscribe). Das bedeutet: Komponenten reden nicht direkt miteinander, sondern senden Events. Vorteil: Lose Kopplung, einfache Erweiterbarkeit, und mehrere Personen können gleichzeitig an verschiedenen Apps arbeiten, ohne sich gegenseitig zu blockieren.

Die drei Kernkomponenten

Komponente	Verantwortung
EventBus	Globaler Nachrichtenkanal – jede App kann publizieren und abonnieren.
WindowManager	Verwaltet alle offenen Fenster, Z-Order, Fokus, Minimieren.
AppRegistry	Kennt alle verfügbaren Apps und kann sie auf Anfrage starten.
VirtualFileSystem	Simuliertes Dateisystem als JSON-Baum, von Apps gemeinsam genutzt.
SettingsService	Lädt/speichert User-Einstellungen, Theme, Fensterpositionen.

Datenflussbeispiel

Beispiel: Nutzer doppelklickt im Datei-Explorer auf *notes.txt*:

- FileExplorer publiziert Event `file.open` mit Pfad.
- AppRegistry abonniert dieses Event und entscheidet: *notes.txt* → NotesApp.
- AppRegistry erzeugt eine NotesApp-Instanz und übergibt sie an den WindowManager.
- WindowManager öffnet das Fenster und gibt ihm den Fokus.

Wichtig: FileExplorer kennt NotesApp nicht – die Kopplung läuft über Strings und den EventBus. Das ist **Clean Code**: Dependency Inversion.

3. Ordnerstruktur

Diese Struktur wird in Phase 1 angelegt und ist verbindlich. Jede neue App bekommt ihren eigenen Unterordner in apps/.

```
cyberdesk/
├── main.py                # Einstiegspunkt – startet QApplication, lädt Desktop
├── README.md              # Setup, Start, Coding Rules
├── requirements.txt        # PyQt6, pytest, etc.
├──
├── core/                  # Kernel-Logik (kein UI hier!)
│   ├── __init__.py
│   ├── event_bus.py       # Publish/Subscribe-Mechanismus
│   ├── window_manager.py  # Verwaltet offene Fenster
│   ├── app_registry.py    # Kennt alle Apps, startet sie
│   └── logger.py          # Zentraler Logger
│   └──
├── ui/                    # Wiederverwendbare UI-Bausteine
│   ├── __init__.py
│   ├── desktop.py         # Hauptbildschirm mit Hintergrund + Icons
│   ├── taskbar.py         # Untere Leiste mit offenen Fenstern
│   ├── start_menu.py      # Startmenü
│   ├── window_frame.py    # Custom Fensterrahmen (Cyberpunk-Look)
│   └── widgets/           # Buttons, Inputs, etc. mit Theme
│   └──
├── apps/                  # Jede App ist ein eigenes Modul
│   ├── __init__.py
│   ├── base_app.py        # Abstrakte Basisklasse für alle Apps
│   ├── terminal/
│   │   ├── __init__.py
│   │   ├── terminal_app.py
│   │   └── commands.py    # ls, cd, cat, fortune, ...
│   ├── file_explorer/
│   ├── notes/
│   ├── netrunner/         # "Browser" mit statischen Dystopie-Seiten
│   └── sysmon/            # Systemmonitor (Fake-Werte)
│   └──
├── services/              # Geteilte Services
│   ├── __init__.py
│   ├── virtual_fs.py      # Simuliertes Dateisystem
│   ├── settings.py        # User-Settings laden/speichern
│   └── sound.py           # Soundeffekte
│   └──
├── data/                  # Persistente Daten (JSON)
│   ├── filesystem.json    # Initialer FS-Zustand
│   ├── settings.json
│   └── lore/              # Texte für die Dystopie-Welt
│   └──
├── assets/                # Statische Ressourcen
│   ├── fonts/             # z.B. Share Tech Mono, VT323
│   ├── images/
│   ├── sounds/
│   ├── styles/
│   └── cyberpunk.qss       # Qt-Stylesheet (wie CSS)
│   └──
└── tests/                 # Unit-Tests (pytest)
    ├── test_event_bus.py
    ├── test_window_manager.py
    └── test_virtual_fs.py
```

4. Rollen & Rotation

Jeder im Team durchläuft mehrere Rollen, damit am Ende jeder in allen Bereichen gearbeitet hat. Personen werden mit A, B, C, D, E bezeichnet – ihr verteilt die Namen beim Kickoff.

Rollendefinitionen

Rolle	Aufgaben
Architektur/Core	EventBus, WindowManager, AppRegistry, Kern-Refactorings.
UI/Design	QSS-Theme, Fensterrahmen, Desktop-Hintergrund, Animationen.
Apps/Features	Konkrete Apps bauen (Terminal, Explorer, Notes, etc.).
Integration/Tests	Apps zusammenführen, Bugs finden, Unit-Tests schreiben.
Doku/Easter Eggs	README, Codekommentare, Easter Eggs, Präsentation.

Rotationsmatrix

In jeder der 4 Phasen wechselt jeder die Rolle. So sieht jeder mindestens 4 von 5 Bereichen:

Bereich	Phase 1	Phase 2	Phase 3	Phase 4
Architektur/Core	A	B	C	A
UI/Design	B	C	D	E
Apps/Features	C	D	E	B
Integration/Tests	D	E	A	C
Doku/Easter Eggs	E	A	B	D

Hinweis: Pro Phase gibt es immer einen **Architektur-Verantwortlichen**, der Code-Reviews abnickt. So bleibt die Codequalität auch bei Rotation hoch.

5. Phasen-Ablaufplan (64 Stunden)

Phase 1 // FUNDAMENT (Stunden 0–16)

Ziel am Ende der Phase: Ein leeres Cyberpunk-Fenster lässt sich auf dem Desktop draggen, schließen und minimieren. Architektur steht.

- Kickoff-Meeting (1h): Rollen verteilen, Namen für A–E festlegen.
- Git-Repo aufsetzen, .gitignore, README, requirements.txt.
- Klassendiagramm zeichnen (Architektur-Verantwortlicher A).
- Implementierung EventBus, WindowManager, AppRegistry, BaseApp.
- Erstes Test-Fenster, das auf Desktop draggable ist.
- QSS-Grundgerüst (Farben, Fonts, eine erste Button-Klasse).
- Coding-Standards-Doku im README.

Phase 2 // APPS & LOOK (Stunden 16–32)

Ziel: Fünf Apps existieren grundlegend, jede in ihrem Fenster lauffähig. Der Cyberpunk-Look ist erkennbar.

- Pro Person eine App: Terminal, FileExplorer, Notes, NetRunner, SysMon.
- Jede App erbt von BaseApp – keine direkten Querbezüge.
- QSS-Theme final: Neon-Cyan/Magenta, Scanlines, Monospace-Fonts.
- Window-Frame mit Cyberpunk-Look (Titlebar, Buttons, Border).
- Erste Easter-Egg-Ideen sammeln (im Issue-Tracker).
- Zwischen-Review nach Phase 2: Demo aller Apps, Code-Review.

Phase 3 // INTEGRATION (Stunden 32–48)

Ziel: Die Apps reden über den EventBus miteinander. Drag & Drop funktioniert. Taskleiste, Startmenü, Lockscreen sind da.

- Drag & Drop: Dateien vom Explorer ins Terminal/Notes ziehen.
- Taskleiste mit offenen Fenstern (Klick = fokussieren).
- Startmenü mit allen verfügbaren Apps.
- Lockscreen beim Start (Passwort: irgendwas Dystopisches).
- Persistenz: Fensterpositionen, Notes, VFS-Zustand in JSON.
- Sounds einbauen: Boot, Klick, Fehler, Notification.

Phase 4 // POLISH & DEFENSE (Stunden 48–64)

Ziel: Bugfrei, Easter Eggs drin, Doku komplett, Präsentation steht.

- Bug-Fixing-Tag (alle, parallel).
- Easter Eggs implementieren (siehe Kapitel 13).
- Unit-Tests für Core (EventBus, WindowManager, VirtualFS).
- README final, Code-Kommentare prüfen.

- Präsentation und Live-Demo vorbereiten.
- Abnahme-Checkliste durchgehen (Kapitel 16).

6. Detaillierter Stunden-Plan

Wenn ihr z.B. 2x4h pro Woche arbeitet, entspricht das 8 Sitzungen pro Phase. Plant pro Sitzung einen klaren Output ein.

Std.	Inhalt	Verantwortlich
0–2	Kickoff, Git-Setup, Klassendiagramm	Alle
2–6	Core: EventBus + WindowManager	Arch + 1 Partner
6–10	BaseApp + erstes Testfenster + QSS-Grundgerüst	Arch + UI
10–14	AppRegistry, Desktop-Skelett, README, Tests-Setup	Alle
14–16	Phase-1-Review, Demo des leeren Fensters	Alle
16–22	Apps starten: Terminal, Explorer, Notes parallel	Apps-Team
22–28	Apps: NetRunner, SysMon, plus QSS-Theme final	Apps + UI
28–32	Phase-2-Review: Demo aller Apps, Code-Review	Alle
32–38	Drag & Drop + Taskleiste + Startmenü	Apps + Integration
38–44	VirtualFS-Persistenz, Settings-Service, Sounds	Integration
44–48	Lockscreen, Boot-Sequenz, Phase-3-Review	UI + Alle
48–54	Bug-Fixing, Unit-Tests, Refactoring	Tests-Team
54–60	Easter Eggs, Polish, Animationen	Doku/Easter + UI
60–64	Doku final, Präsentation, Abnahme-Check	Alle

7. Coding Standards & Clean Code

Diese Regeln sind verbindlich für alle. Sie werden im README verlinkt und bei Code-Reviews kontrolliert.

Grundregeln

- **PEP8:** Einrückung 4 Spaces, max. 100 Zeichen pro Zeile.
- **Type Hints:** Jede Funktion hat Typannotationen.
- **Docstrings:** Jede öffentliche Klasse und Funktion bekommt einen Docstring (Google- oder NumPy-Style).
- **Naming:** snake_case für Funktionen/Variablen, PascalCase für Klassen, UPPER_CASE für Konstanten.
- **Sprechende Namen:** `open_window(app_id)` statt `do(x)`.
- **Funktionen kurz:** Idealerweise < 20 Zeilen, eine Verantwortung.
- **Keine Magic Numbers:** Konstanten benennen.
- **Imports:** Sortiert: `stdlib` → `third-party` → eigene Module.
- **Logging statt print:** `logger.info(...)`, nicht `print(...)`.
- **Kein Vibe Coding:** Wer Code commitet, muss erklären können, was er tut. KI darf assistieren, aber nie blind kopiert werden.

SOLID-Prinzipien (anwendbar in unserem Projekt)

- **Single Responsibility:** Eine Klasse = eine Aufgabe. `WindowManager` verwaltet keine Settings.
- **Open/Closed:** `BaseApp` ist offen für Erweiterung, nicht für Änderung.
- **Liskov Substitution:** Jede App muss überall einsetzbar sein, wo `BaseApp` erwartet wird.
- **Interface Segregation:** Keine fetten Basisklassen mit ungenutzten Methoden.
- **Dependency Inversion:** Apps kennen nur den `EventBus`, keine konkreten anderen Apps.

Werkzeuge

- `black` – automatischer Formatter (vor jedem Commit).
- `ruff` oder `flake8` – Linter.
- `mypy` – Type-Checker (optional, aber empfohlen).
- `pytest` – Tests.

8. Git-Workflow

Wir nutzen den **Feature-Branch-Workflow**. Niemand pusht direkt auf main. Jede neue Funktion bekommt einen eigenen Branch und wird per Pull Request gemerged.

Branch-Namen

- feature/<app>-<kurzbeschreibung> z.B. feature/terminal-ls-command
- bugfix/<beschreibung>
- docs/<was>

Commit-Messages

Wir nutzen **Conventional Commits**:

```
feat(terminal): add ls command with hidden file support
fix(window_manager): correct z-order after minimize
docs(readme): add setup instructions for Windows
refactor(event_bus): simplify subscribe logic
test(virtual_fs): add tests for path resolution
```

Pull-Request-Regeln

- Mindestens **ein Reviewer** aus dem Team muss zustimmen.
- Lokal müssen black, Linter und Tests grün sein.
- PR-Beschreibung: Was, Warum, Wie getestet.
- Bei Konflikten: rebasen, nicht mergen.

9. Starter-Code: Kernsystem

Dieser Code ist die Grundlage für Phase 1. Er ist bewusst minimal und sauber – ihr erweitert ihn, ohne ihn zu "verbiegen". Alle Kernkomponenten leben in `core/`.

`core/event_bus.py`

Der EventBus ist das Herz der Kommunikation. Komponenten abonnieren Events und reagieren, wenn sie publiziert werden.

```
"""Globaler Event-Bus für lose Kopplung zwischen Komponenten."""

from __future__ import annotations
from collections import defaultdict
from typing import Callable, Any
import logging

logger = logging.getLogger(__name__)

# Typ-Alias für Lesbarkeit
EventHandler = Callable[[dict[str, Any]], None]

class EventBus:
    """Einfaches Publish/Subscribe-System.

    Beispiel:
        bus = EventBus()
        bus.subscribe("file.open", lambda payload: print(payload["path"]))
        bus.publish("file.open", {"path": "/home/user/notes.txt"})
    """

    def __init__(self) -> None:
        # Mapping: event_name -> Liste von Handlern
        self._subscribers: dict[str, list[EventHandler]] = defaultdict(list)

    def subscribe(self, event_name: str, handler: EventHandler) -> None:
        """Registriert einen Handler für ein Event."""
        self._subscribers[event_name].append(handler)
        logger.debug("Subscribed handler to %s", event_name)

    def unsubscribe(self, event_name: str, handler: EventHandler) -> None:
        """Entfernt einen Handler."""
        if handler in self._subscribers[event_name]:
            self._subscribers[event_name].remove(handler)

    def publish(self, event_name: str, payload: dict[str, Any] | None = None) -> None:
        """Sendet ein Event an alle Abonnenten."""
        payload = payload or {}
        logger.debug("Publishing %s with %s", event_name, payload)
        for handler in list(self._subscribers[event_name]):
            try:
                handler(payload)
            except Exception: # noqa: BLE001
                logger.exception("Handler für %s ist fehlgeschlagen", event_name)

# Singleton-Instanz für globalen Zugriff
event_bus = EventBus()
```

`apps/base_app.py`

Abstrakte Basisklasse, von der jede App erbt. Garantiert, dass jede App ein Fenster, einen Titel und ein Icon hat.

```
"""Abstrakte Basisklasse für alle CyberDesk-Apps."""

from __future__ import annotations
from abc import abstractmethod
from PyQt6.QtWidgets import QWidget
from PyQt6.QtGui import QIcon

from core.event_bus import event_bus

class BaseApp(QWidget):
    """Basis für jede App im CyberDesk-System.

    Jede konkrete App muss:
    - app_id und app_title als Klassenattribute setzen
    - build_ui() implementieren
    """

    app_id: str = "base"
    app_title: str = "Untitled"
    app_icon: str = "assets/images/default_icon.png"

    def __init__(self, parent: QWidget | None = None) -> None:
        super().__init__(parent)
        self.setWindowTitle(self.app_title)
        self.setWindowIcon(QIcon(self.app_icon))
        self.bus = event_bus
        self.build_ui()
        self.connect_events()

    @abstractmethod
    def build_ui(self) -> None:
        """Hier wird das UI der App aufgebaut."""

    def connect_events(self) -> None:
        """Optional: hier kann die App Events abonnieren."""

    def on_close(self) -> None:
        """Wird vom WindowManager beim Schließen aufgerufen."""
        self.bus.publish("app.closed", {"app_id": self.app_id})
```

core/window_manager.py

Der WindowManager kennt alle offenen Fenster und verwaltet Fokus, Z-Order und Minimieren.

```
"""Verwaltet alle offenen Fenster auf dem CyberDesk."""

from __future__ import annotations
from PyQt6.QtWidgets import QWidget
from typing import Dict
import uuid
import logging

from core.event_bus import event_bus

logger = logging.getLogger(__name__)

class WindowManager:
    """Hält eine Referenz auf jedes offene Fenster.
```

Identifiziert Fenster über eine eindeutige UUID, damit zwei Instanzen derselben App gleichzeitig laufen können.

```
"""
def __init__(self, desktop: QWidget) -> None:
    self._desktop = desktop
    self._windows: Dict[str, QWidget] = {}
    event_bus.subscribe("window.close", self._on_close_request)

def open_window(self, app: QWidget) -> str:
    """Öffnet ein neues Fenster und gibt die Window-ID zurück."""
    window_id = str(uuid.uuid4())
    self._windows[window_id] = app
    app.setParent(self._desktop)
    app.show()
    app.raise_() # nach vorne in der Z-Order
    event_bus.publish("window.opened",
                      {"window_id": window_id, "title": app.windowTitle()})
    logger.info("Opened window %s (%s)", window_id, app.windowTitle())
    return window_id

def close_window(self, window_id: str) -> None:
    """Schließt ein Fenster."""
    window = self._windows.pop(window_id, None)
    if window is None:
        logger.warning("Tried to close unknown window %s", window_id)
        return
    if hasattr(window, "on_close"):
        window.on_close()
    window.close()
    window.deleteLater()
    event_bus.publish("window.closed", {"window_id": window_id})

def _on_close_request(self, payload: dict) -> None:
    """Reagiert auf Event window.close."""
    window_id = payload.get("window_id")
    if window_id:
        self.close_window(window_id)

@property
def open_count(self) -> int:
    """Anzahl aktuell offener Fenster."""
    return len(self._windows)

```

core/app_registry.py

Die AppRegistry kennt alle verfügbaren Apps und kann sie auf Anfrage starten. So müssen sich Apps nicht gegenseitig importieren.

```
"""Registry, die alle bekannten Apps kennt und sie starten kann."""
```

```
from __future__ import annotations
from typing import Type
from apps.base_app import BaseApp
from core.window_manager import WindowManager
from core.event_bus import event_bus
```

```
class AppRegistry:
    """Hält ein Mapping von app_id -> App-Klasse."""

    def __init__(self, window_manager: WindowManager) -> None:
        self._apps: dict[str, Type[BaseApp]] = {}
        self._wm = window_manager
```

```

        event_bus.subscribe("app.launch", self._on_launch)

    def register(self, app_class: Type[BaseApp]) -> None:
        """Registriert eine neue App-Klasse."""
        self._apps[app_class.app_id] = app_class

    def launch(self, app_id: str) -> None:
        """Startet eine App anhand ihrer ID."""
        app_class = self._apps.get(app_id)
        if app_class is None:
            raise KeyError(f"Unbekannte App: {app_id}")
        instance = app_class()
        self._wm.open_window(instance)

    def available_apps(self) -> list[str]:
        """Gibt eine Liste aller registrierten App-IDs zurück."""
        return sorted(self._apps.keys())

    def _on_launch(self, payload: dict) -> None:
        """Reagiert auf Event app.launch."""
        app_id = payload.get("app_id")
        if app_id:
            self.launch(app_id)

```

main.py (Einstiegspunkt)

```

"""CyberDesk - Einstiegspunkt."""

from __future__ import annotations
import sys
import logging

from PyQt6.QtWidgets import QApplication

from core.window_manager import WindowManager
from core.app_registry import AppRegistry
from ui.desktop import Desktop

# Apps importieren (registrieren sich indirekt)
from apps.terminal.terminal_app import TerminalApp
# from apps.file_explorer.explorer_app import FileExplorerApp
# from apps.notes.notes_app import NotesApp

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")

def main() -> int:
    """Startet die CyberDesk-Anwendung."""
    app = QApplication(sys.argv)

    # Stylesheet laden
    with open("assets/styles/cyberpunk.qss", encoding="utf-8") as f:
        app.setStyleSheet(f.read())

    desktop = Desktop()
    window_manager = WindowManager(desktop)
    registry = AppRegistry(window_manager)

    # Apps registrieren
    registry.register(TerminalApp)
    # registry.register(FileExplorerApp)
    # registry.register(NotesApp)

```

```
desktop.show_fullscreen()  
return app.exec()  
  
if __name__ == "__main__":  
    sys.exit(main())
```

10. App-Spezifikationen

Jede App ist ein eigenes Modul unter apps/. Diese Specs sind die Mindestanforderungen.

Terminal

Eine simulierte Shell mit eigenen Befehlen. Liest aus VirtualFS, schreibt zurück.

- Befehle: ls, cd, cat, echo, clear, help, whoami, date, fortune.
- Fake-Befehle: hack, decrypt, scan, jack-in (Cyberpunk-Flavor).
- Command-History mit Pfeiltasten.
- Autovervollständigung von Pfaden mit Tab (Stretch Goal).
- Easter-Egg-Befehle (siehe Kapitel 13).

File Explorer

Zeigt das VirtualFS als Baum/Liste. Drag & Drop zwischen Ordnern und in andere Apps.

- Zwei-Panel-Layout: Baum links, Inhalt rechts.
- Doppelklick öffnet Datei mit passender App (über EventBus).
- Rechtsklick-Menü: Umbenennen, Löschen, Eigenschaften.
- Verstecktes Verzeichnis `.secret/` für Lore.

Notes

Einfacher Text-Editor mit Speichern/Laden ins VirtualFS.

- QPlainTextEdit als Editor.
- Speichern (Strg+S), Öffnen, Neu.
- Status-Leiste: Zeile/Spalte, Zeichenanzahl.
- Auto-Save alle 30 Sekunden.

NetRunner (Browser)

Ein fiktiver Browser, der nur Seiten aus data/lore/ anzeigt.

- Adressleiste akzeptiert URLs wie `net://corp/aurora`.
- Statische HTML-Dateien werden geladen.
- Fake-Loading-Animation, gelegentlich "Connection lost".
- Hyperlinks führen zu anderen Lore-Seiten.

SysMon (Systemmonitor)

Zeigt Fake-CPU/RAM/Netzwerk-Werte mit hübschen Graphen.

- Zufalls-Daten, die wie echte Live-Werte aussehen.
- QChart oder selbstgemalter QPainter-Graph.
- Liste der "laufenden Prozesse" – das sind die offenen Apps.
- Easter Egg: ein Prozess heißt "netrunner.exe" und nutzt 99% CPU.

11. Datenmodelle & Persistenz

Alle Zustandsdaten werden als JSON in data/ abgelegt. Beim Start lädt der SettingsService die Daten, beim Beenden werden sie zurückgeschrieben.

VirtualFS – Struktur

```
{
  "/": {
    "type": "dir",
    "children": {
      "home": {
        "type": "dir",
        "children": {
          "user": {
            "type": "dir",
            "children": {
              "notes.txt": {
                "type": "file",
                "content": "Auftrag: Daten von Aurora-Corp extrahieren."
              },
              ".secret": {
                "type": "dir",
                "hidden": true,
                "children": {
                  "manifest.txt": {
                    "type": "file",
                    "content": "Sie sehen uns nicht. Wir sind das Signal."
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Settings – Struktur

```
{
  "theme": "neon-magenta",
  "username": "V",
  "boot_sound": true,
  "last_open_windows": [
    {"app_id": "terminal", "x": 120, "y": 80, "w": 600, "h": 400},
    {"app_id": "notes", "x": 750, "y": 80, "w": 400, "h": 500}
  ]
}
```

12. Cyberpunk Look & Feel

Farbpalette (verbindlich)

Name	Hex	Verwendung
Neon Cyan	#00FFD1	Akzente, Hover, Linien
Neon Magenta	#FF2BD6	Titles, wichtige Buttons
Toxic Yellow	#F2FF00	Warnungen, Easter-Egg-Highlights
Deep Background	#0A0A14	Desktop-Hintergrund
Panel	#1A1A2E	Fensterhintergründe
Text Main	#E0E0F0	Standardtext
Text Dim	#7A7A8C	Sekundärtext

Typografie

- **Headings:** "Share Tech Mono" oder "VT323" (Google Fonts, frei nutzbar).
- **Body:** "JetBrains Mono" oder "Fira Code".
- Niemals Times New Roman oder Arial – das killt die Atmosphäre.

Effekte (subtil halten!)

- Glow auf Buttons beim Hover (QSS border + box-shadow Workaround).
- Leichte Scanlines als Overlay über den Desktop (transparentes PNG).
- Beim Boot: kurze Glitch-Animation mit zufälligem Text.
- Tastatur-Sound beim Tippen im Terminal (sehr leise).

QSS-Beispiel (Fragment)

```
/* assets/styles/cyberpunk.qss */

QWidget {
    background-color: #1A1A2E;
    color: #E0E0F0;
    font-family: "JetBrains Mono";
    font-size: 11pt;
}

QPushButton {
    background-color: #0A0A14;
    color: #00FFD1;
    border: 1px solid #00FFD1;
    padding: 6px 14px;
}

QPushButton:hover {
    background-color: #00FFD1;
    color: #0A0A14;
}

QLineEdit, QPlainTextEdit {
    background-color: #0A0A14;
    color: #00FFD1;
    border: 1px solid #FF2BD6;
```

```
    selection-background-color: #FF2BD6;  
}
```

13. Easter Eggs

Easter Eggs sind das, was eure Demo unvergesslich macht. Plant sie früh, baut sie aber erst in Phase 4 vollständig ein. Jeder im Team bringt mindestens ein Easter Egg ein.

Trigger	Effekt
Terminal: <code>sudo rm /dev/fd0</code>	Fake-Glitch über den ganzen Bildschirm, dann "Nice try, choomba."
Konami-Code (↑↑↓↓←→←→BA)	Retro-Modus: alles wird grün-auf-schwarz, alte CRT-Optik.
Datei <code>/home/user/.Secrets/secret.txt</code>	Speitgreifereinstellung öffnet versteckte Lore-Seiten im NetRunner frei.
Systemzeit 23:59	"SYSTEM ANOMALY DETECTED"-Popup mit zufälligem Cyberpunk-Zitat.
App-Icon 7× schnell klicken	Credits laufen als Hacker-Text durch das Terminal.
Terminal: <code>jack-in</code>	Vollbild-Animation "Connecting to the Net...", danach versteckte Seite.
NetRunner: URL <code>netrunner://</code>	Bildschirm wird komplett schwarz, nur ein blinkender Cursor.
SysMon: Prozess <code>kernel</code>	Kurz ein Bluescreen-Effekt, dann normaler Betrieb.
Notes: Text "wake up" eingeben	Notes-Hintergrund wird kurz rot, dann normal.
10 Fenster gleichzeitig offen	Achievement-Popup: "Multitasker of Night City".

14. Tests & Qualitätssicherung

Wir testen nicht das UI direkt, sondern die Logik dahinter. Faustregel: **Was ohne Qt funktioniert, wird mit pytest getestet.**

Was wird getestet?

- EventBus: Subscribe, Publish, Unsubscribe, Exception-Handling.
- VirtualFS: Pfade auflösen, Datei lesen/schreiben, Ordner erstellen.
- Terminal-Commands: ls, cd, cat isoliert testen.
- SettingsService: Laden, Speichern, Defaults.

Beispiel-Test (pytest)

```
"""tests/test_event_bus.py"""

from core.event_bus import EventBus

def test_subscribe_and_publish_delivers_payload() -> None:
    bus = EventBus()
    received: list[dict] = []
    bus.subscribe("test.event", lambda payload: received.append(payload))

    bus.publish("test.event", {"value": 42})

    assert received == [{"value": 42}]

def test_unsubscribe_stops_delivery() -> None:
    bus = EventBus()
    received: list[dict] = []

    def handler(payload: dict) -> None:
        received.append(payload)

    bus.subscribe("test.event", handler)
    bus.unsubscribe("test.event", handler)
    bus.publish("test.event", {"value": 1})

    assert received == []

def test_handler_exception_does_not_break_other_handlers() -> None:
    bus = EventBus()
    received: list[dict] = []

    def bad_handler(_: dict) -> None:
        raise RuntimeError("boom")

    bus.subscribe("test.event", bad_handler)
    bus.subscribe("test.event", lambda p: received.append(p))

    bus.publish("test.event", {"ok": True})

    assert received == [{"ok": True}]
```

15. Risiken & Notfallplan

Risiko	Gegenmaßnahme
Scope-Creep: zu viele Features	Feature-Liste am Anfang einfrieren. Neue Ideen ins Backlog, nicht sofort einbauen.
Merge-Konflikte durch parallele Arbeit	Strikter Feature-Branch-Workflow, kleine PRs, früh mergen.
Eine Person blockiert (krank/abwesend)	Durch Rollenrotation kann jeder einspringen. Wissen ist nie in einer Person.
Cyberpunk-Look wird hässlich	Einen Designverantwortlichen pro Phase – Stylesheet zentral, nicht überall verstreut.
Performance-Probleme bei vielen Profilen	Profiler nutzen (cProfile). Animationen reduzieren, wenn nötig.
KI generiert unsauberen Code	Code-Review-Pflicht. Wer commitet, muss erklären können. Kein Vibe Coding.
Drag & Drop ist instabil	Früh prototypisieren (Phase 1), nicht erst in Phase 3.

16. Abnahme-Checkliste

Vor der Präsentation geht das ganze Team diese Liste gemeinsam durch.

- ■ App startet ohne Fehler aus dem Terminal.
- ■ Boot-Sequenz + Lockscreen laufen.
- ■ Mindestens 5 Apps sind funktionsfähig.
- ■ Drag & Drop funktioniert zwischen Explorer und mindestens einer App.
- ■ Fenster lassen sich verschieben, schließen, minimieren, fokussieren.
- ■ Taskleiste zeigt offene Fenster korrekt.
- ■ Settings (Fensterpositionen, Notes) bleiben nach Neustart erhalten.
- ■ Cyberpunk-Look ist konsistent (keine grauen Standard-Buttons).
- ■ Mindestens 5 Easter Eggs sind dokumentiert und funktionieren.
- ■ README erklärt Setup und Bedienung.
- ■ Code ist mit black formatiert, Linter ist grün.
- ■ Mindestens 10 Unit-Tests laufen grün.
- ■ Jeder im Team kann jeden Bereich kurz erklären.
- ■ Präsentation ist eingeübt, Live-Demo getestet.

Wenn alle Punkte abgehakt sind: Ihr seid bereit. Geht da raus und zeigt Night City, wie ein Desktop auszusehen hat.